Doubly Linked List – Traverse In Order And Time Complexity

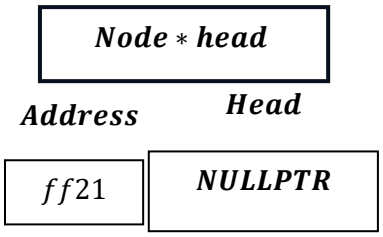
Program :

```
void traverseInOrder()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    cout << "List (Forward): NULL <- ";
    while (temp !=nullptr)
    {
        cout << temp->data << " <-> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
…..
case 10:
   traverseInOrder();
    break;
```

Program Analysis

A. When LIST is Empty

```
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return ;
    }
```

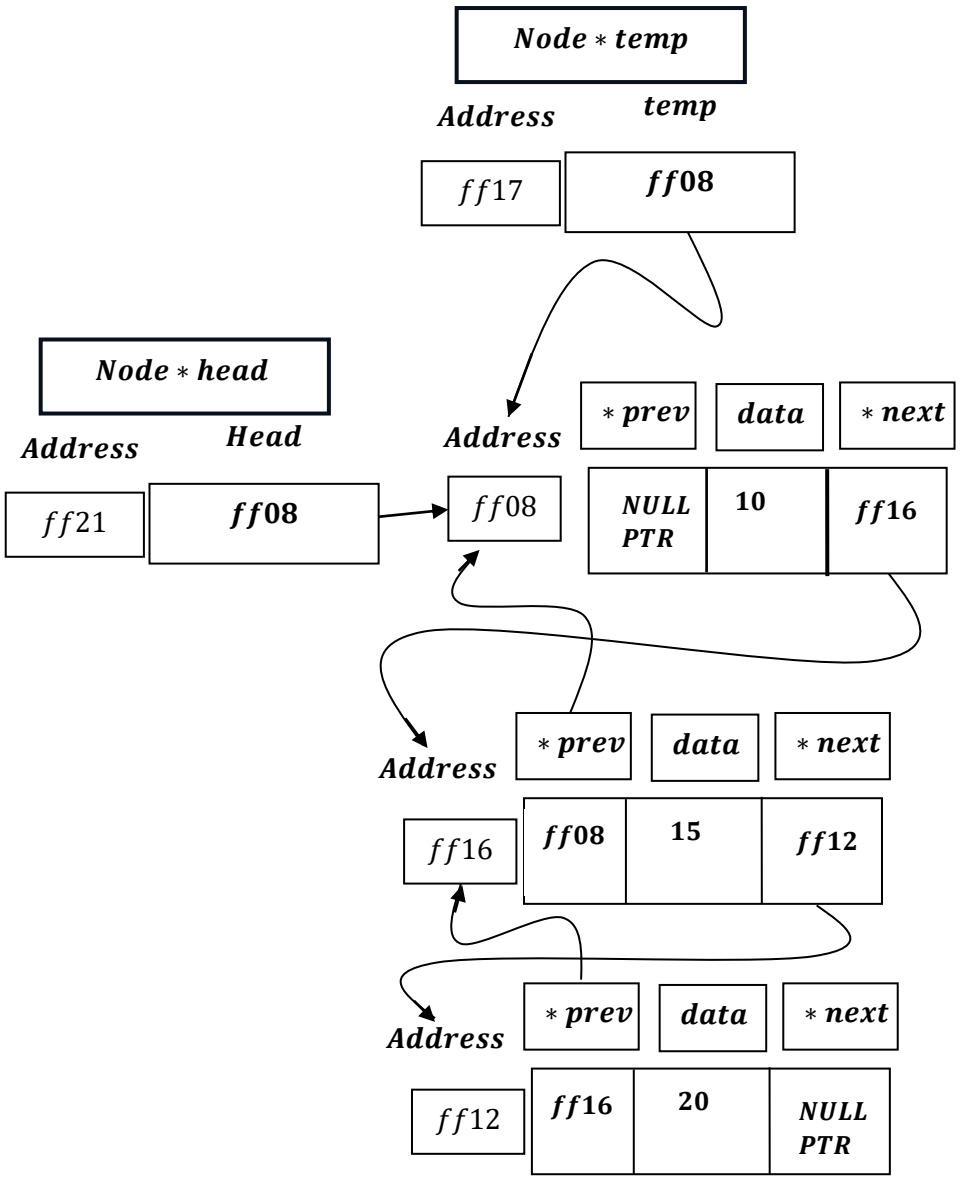


Then, it will print statement : `` List is empty.``

And, return statement will exit from the function.

B. When LIST is not Empty

```
Node *temp = head;
```

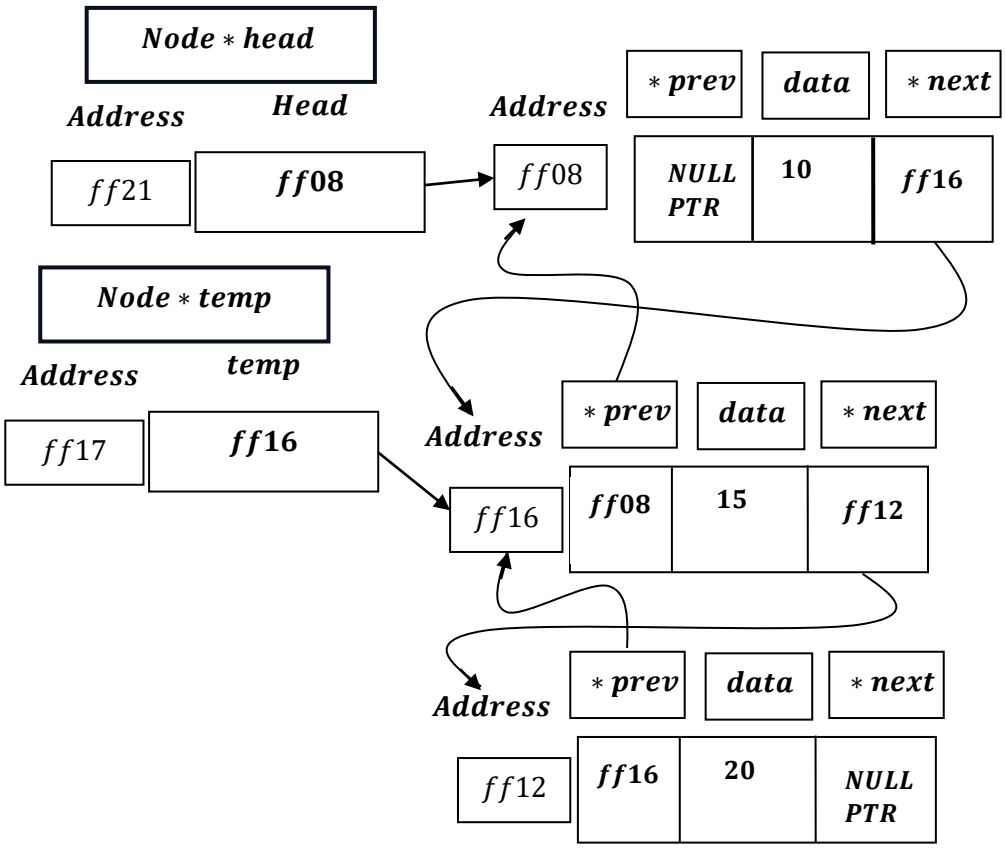


```
    cout << "List (Forward): NULL <- ";
    while (temp !=nullptr)
    {
        cout << temp->data << " <-> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
```

$\boldsymbol{Temp \neq NULLPTR\ is\ true\ , hence:}$

$\boldsymbol{cout \ll temp \rightarrow data \ll <->\ ; \Longrightarrow Output: NULL < -10 < ->}$

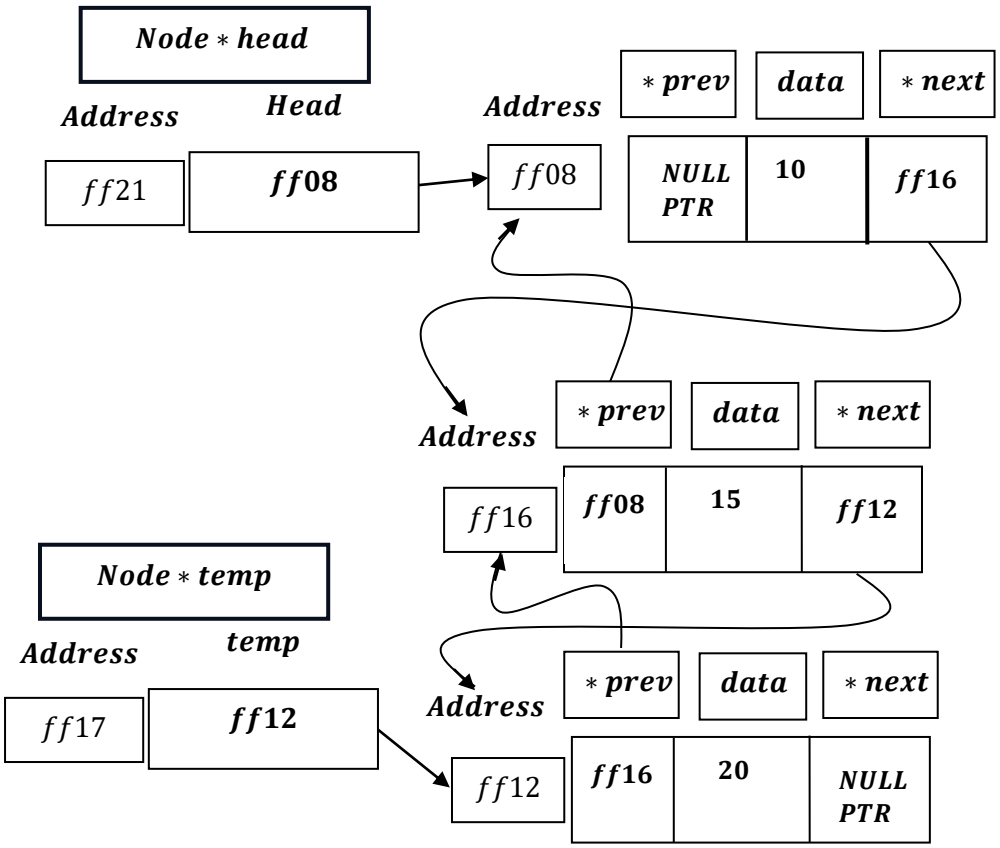
$\boldsymbol{temp = temp \rightarrow next = ff16}$.



$\boldsymbol{Temp \neq NULLPTR\ is\ true\ , hence:}$

$\boldsymbol{cout \ll temp \rightarrow data \ll <->\ ; \Longrightarrow Output: NULL < -10 < -> 15 < ->}$

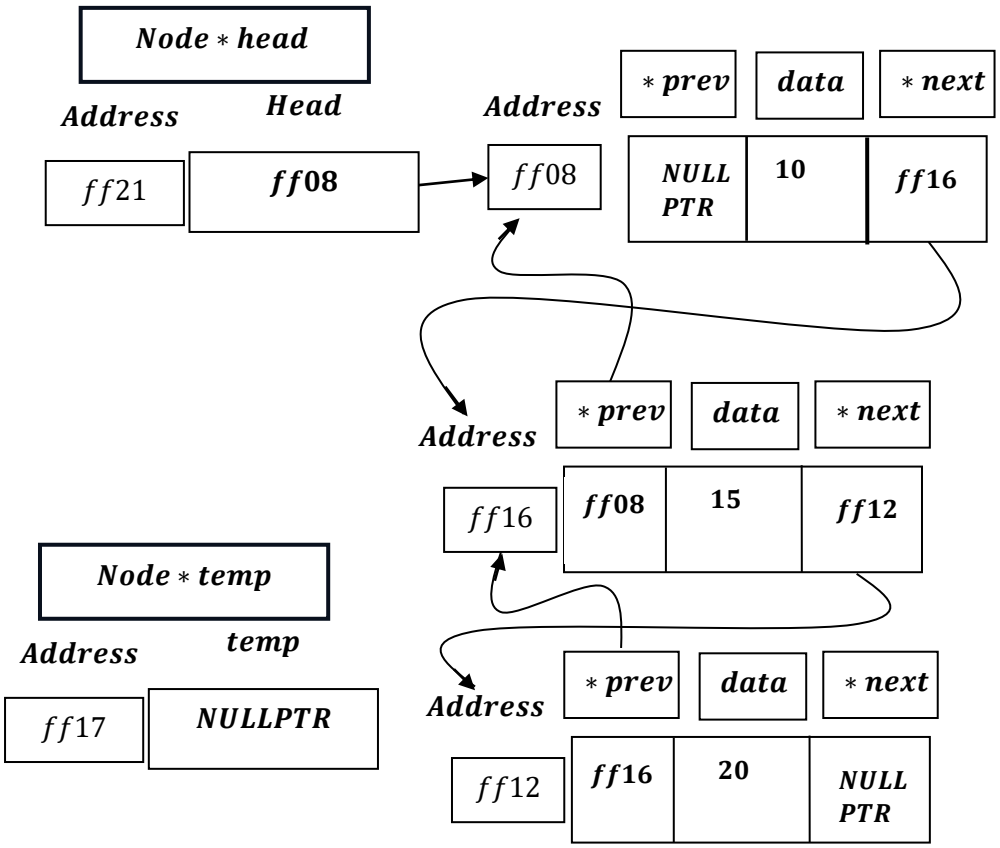
$\boldsymbol{temp = temp \rightarrow next = ff12}$.



$Temp \neq NULLPTR$ *is true* , *hence*:

$cout \ll temp \rightarrow data \ll$ **<->** ; $\Longrightarrow$ *Output*: $NULL < -10 < -> 15 < -> 20 < ->$

$temp = temp \rightarrow next = NULLPTR$.

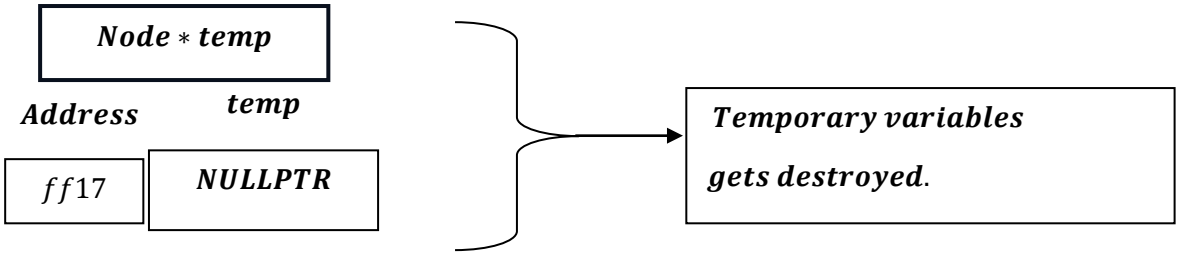


$Now, TEMP = NULLPTR$, *hence while* $(TEMP \neq NULLPTR) \rightarrow$ *condition fails* .

```
    cout << "NULL" << endl;
```

$cout \ll$ "$NULL$" $\ll$ **<->** ; $\Longrightarrow$ *Output*: $NULL < -10 < -> 15 < -> 20 < -> NULL$

As function ends , Temp pointer gets detroyed:



Doubly Linked List – Traverse In Order [Working Summary]:

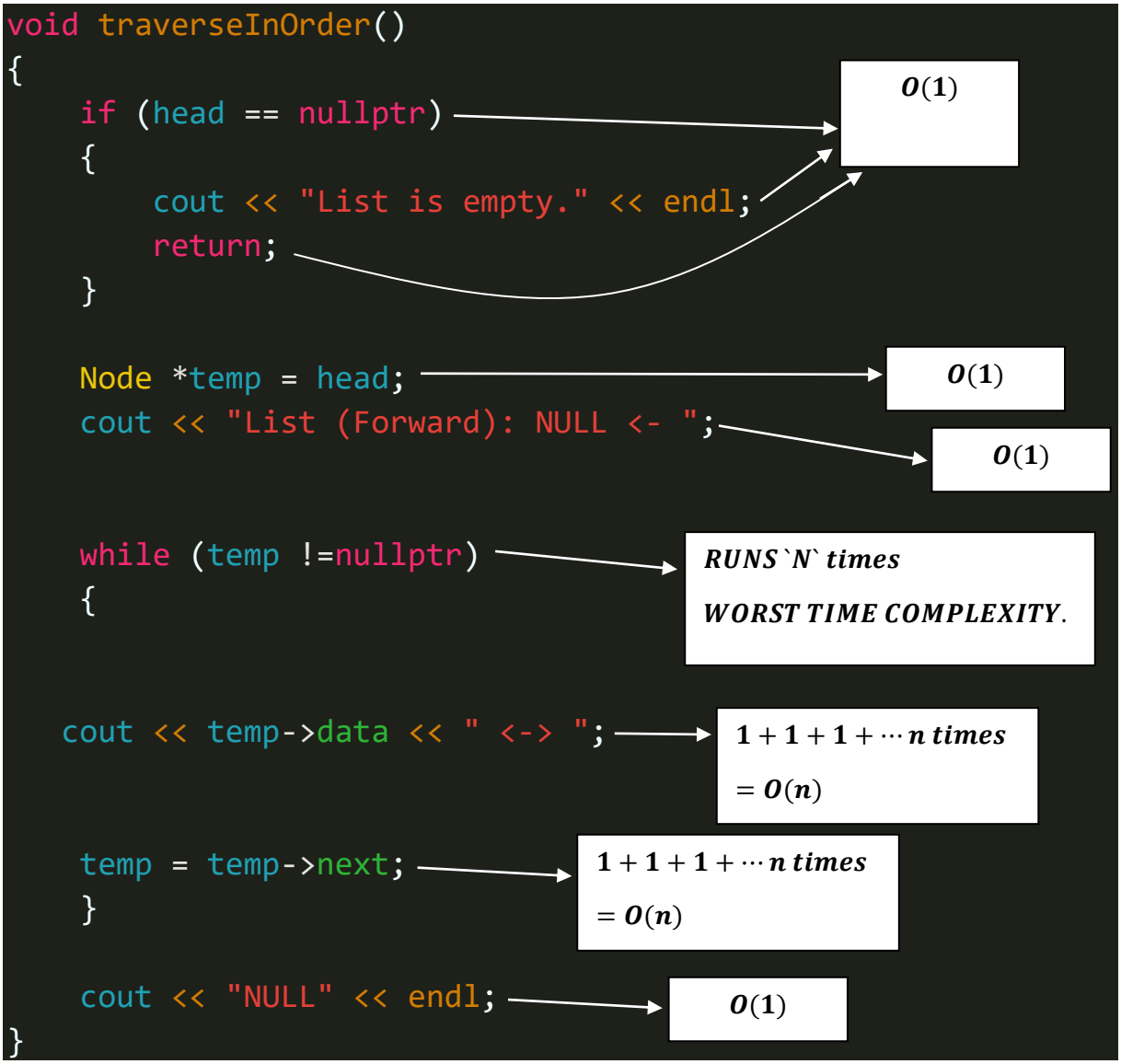
This C++ function, `traverseInOrder`, is designed to print all the elements of a linked list from beginning to end.

Working Summary

1. **Handles Empty List:** It first checks if the list is empty by verifying if the `head` pointer is `nullptr`. If so, it prints "List is empty." and exits.
2. **Starts Traversal:** If the list is not empty, it creates a temporary pointer `temp` initialized to the `head` of the list. It then prints a header string `List (Forward): NULL <-` to indicate the start of the list.
3. **Iterates and Prints:** The function enters a `while` loop that continues as long as `temp` has not reached the end of the list (`temp != nullptr`). In each iteration:
    - It prints the `data` of the current node.
    - It prints `<->` to visually represent the links between nodes.
    - It moves the `temp` pointer to the next node (`temp = temp->next`).
4. **Finishes Output:** Once the loop completes, it prints `NULL` to signify the end of the list, completing the visual representation. For a list containing 10, 20, and 30, the output would look like this:

5. `List (Forward): NULL <- 10 <-> 20 <-> 30 <-> NULL`

Time Complexity Of Traverse In Order Function Module



Program Analysis

Absolute Best Case:

$$\boldsymbol{When\ List\ is\ Empty\ (head = NULLPTR)}$$

$\rightarrow$ ***If*** $\boldsymbol{head = nullptr}$ ***, this checking takes constant time. The cout and return statements takes constant time. Hence overall process takes,*** $\boldsymbol{O(1)\ time}$.

Worst Case(When List is not Empty)

$\rightarrow$ $\boldsymbol{Node * temp = * head;}$ ***this assignment takes constant time i.e.*** $\boldsymbol{O(1)}$.

$\rightarrow$ $\boldsymbol{cout \ll "List\ (Forward) :\ NULL\ <-";}$ ***takes constant time to execute i.e.*** $\boldsymbol{O(1)}$.

$\rightarrow$ $\boldsymbol{while(temp\ !=\ nullptr)}$ ***checking becomes true , then:***

$\rightarrow$ $\boldsymbol{cout \ll temp \rightarrow data \ll " <-> ";}$ ***, this statement gets executed:*** $\boldsymbol{1 + 1 + \cdots n\ times = n;}$

$\rightarrow$ $\boldsymbol{temp = temp \rightarrow next;}$ ***such assignment takes,*** $\boldsymbol{1 + 1 + \cdots n\ times = n;}$

Therefore , total loop time complexity: $\boldsymbol{2n}$ ***or*** $\boldsymbol{O(n)}$.

$\rightarrow$ $\boldsymbol{cout \ll "NULL "}$ ***takes constant time to execute i.e.*** $\boldsymbol{O(1)}$.

Hence total time complexity : $\boldsymbol{O(1) + O(1) + O(n) + O(1) = O(n)}$.

Therefore takes $\boldsymbol{O(n)}$ ***time .***

Average Case

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine list get traversed from beginning to end , hence we can say average case is our worst case here i. e. always the entire list is traversed i. e. $O(n) = \Theta(n)$.

This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (***best case***). ***If the list is not empty, the execution time is proportional to the length of the list*** (***worst case***). ***There's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence , Traverse In Order function doesn't have a true average case.***

Note , by relation: $\Omega \leq \Theta \leq O$ ***, we get:***

$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$

Doubly Linked List	***Cases***	***Time Complexity***
Traverse In Order	**1.** ***Best Case***	$\boldsymbol{\Omega(1)}$
	2. ***Worst Case***	$\boldsymbol{O(n)}$
	3. ***Average Case*** $(= \boldsymbol{Worst\ Case})$	$\boldsymbol{\Theta(n)}$
